

(PCC) Principles of Programming Languages

Teaching Scheme

Lectures: 2 Hrs/ Week

Laboratory: 2 Hrs/ Week

Self-Study: 1 Hr/Week

Evaluation Scheme

Theory: MSE:30 Marks, TA:20 Marks ESE:50 Marks

Laboratory: CIE: 100 Marks

Course outcomes

Students will be able to:

1. Explain, compare and discuss different language translation mechanisms.
2. Explain fundamental concepts of different programming paradigms.
3. Discuss and analyze factors that impact implementation of different programming language concepts and tradeoffs involved.
4. Demonstrate ability to write simple programs using exception handling and event driven programming concepts.
5. Suggest a suitable programming paradigm for a given problem.
6. implement the solution for a given problem using different programming paradigms.

Course content

Preliminaries:

[5 Hrs]

Reasons for Studying Concepts of Programming Languages. Programming Domains. Language Evaluation Criteria. Influences on Language Design: computer architecture. Language Categories. Language Design Trade-Offs.

Implementation Methods:

[5 Hrs]

Compilation, Interpretation, Hybrid Implementation, Preprocessors. Programming Environments. Evolution of the Major Programming Languages (Pseudocodes, Assembly, C, C++, FORTRAN, Java). Introduction to assembly language instructions.

Syntax and Semantics Lexical and Syntax Analysis

[8 Hrs]

Names, Bindings, and Scopes, Type checking, Strong Typing. Type Conversions. Short-Circuit Evaluation.

Statement-Level Control Structures Subprograms:

[6 Hrs]

Introduction. Fundamentals of Subprograms. Design Issues for Subprograms. Local Referencing Environments. Parameter- Passing Methods.

Exception Handling:

[6 Hrs]

Basic Concepts. Design Issues. Exception Handlers. Binding Exceptions to Handlers. Event Handling: Basic Concepts. Event handling with Programming Language. Comparison of Exception handling & Event Handling. GUI development

Programming Paradigms:

[10 Hrs]

Introduction to Logic Programming. Introduction. Applications of Logic Programming. Introduction to Functional Programming. Introduction. Introduction to LISP. Garbage Collection Algorithms. A Comparison of Functional and Imperative Languages. Introduction to Procedural Programming. Introduction to Object Oriented Programming

Self-study

[12 Hrs]

Python as a multi-paradigm language for functional and logic programming. Case study of Ada, PERL, Go and

Ruby for their features. Comparison of efficiency of compiled and interpreted languages in the form of assignments.

Textbooks

1. Schesta R., "Concepts Of Programming Languages", 4th Edition, Pearson Education, ISBN-81-7808-161-X
2. T. W. Pratt , "Programming Languages", 4th Edition ,Prentice-Hall Of India, ISBN 9780130287199.

Reference Books

1. Ghezzi C, Milano P, Jazayeri M., "Programming Languages Concepts", 3rd Edition, John Wiley and Sons Pvt. Ltd (WSE), ISBN - 0195113063
2. Michael L. Scott "Programming Language Pragmatics", ELSEVIER Publication, ISBN: 81-8147-370-1
3. Roosta S., "Foundations of Programming Languages", Thomson, Brooke/Cole, ISBN 981-243-141-1
4. Sethi R., "Programming Languages concepts & constructs", 2nd Edition, Pearson Education, ISBN 81 - 7808 - 104 - 0

Web references

1. <https://nptel.ac.in/courses/106102067>

Laboratory assignments

1. **Task 1:** To study various language implementation methods.
 - a. Draw and explain the working of:
 - i. Compiler
 - ii. Interpreter
 - iii. Hybrid Implementation System
 - iv. Preprocessor
 - b. Write a simple C program and observe:
 - i. Source Code
 - ii. Object Code Generation
 - iii. Executable File Generation
 - c. Compare execution time of:
 - i. C Program
 - ii. Python Program for the same logic (e.g., factorial or array sum).
2. **Task 2:** To understand low-level programming concepts.
 - a. Study basic assembly instructions:
 - i. MOV
 - ii. ADD
 - iii. SUB
 - iv. MUL
 - v. DIV
 - vi. JMP
 - b. Write assembly-style algorithms for:
 - i. Addition of two numbers
 - ii. Largest among three numbers

- iii. Sum of first N numbers
- c. Explain how computer architecture influences language design.
- d. Draw CPU-memory interaction diagram.
- 3. Task 3:** To understand the front-end phases of language processing.
 - a. Given a C program, identify:
 - i. Keywords
 - ii. Identifiers
 - iii. Constants
 - iv. Operators
 - v. Special Symbols
 - b. Construct a token table.
 - c. Draw parse trees for:
 - i. $a+b*c$
 - ii. $(a+b)*(c-d)$
 - d. Write a Python program to count:
 - i. Keywords
 - ii. Operators
 - iii. Identifiers from a source file.
- 4. Task 4:** To study variable visibility and type systems.
 - a. Write a C++ program demonstrating:
 - i. Local Scope
 - ii. Global Scope
 - iii. Block Scope
 - b. Demonstrate static binding and dynamic binding using C++ classes.
 - c. Explain strong typing and weak typing with examples.
 - d. Perform explicit and implicit type conversion examples.
- 5. Task 5:** To understand expression evaluation mechanisms.
 - a. Write programs demonstrating:
 - i. Logical AND (&&)
 - ii. Logical OR (||)
 - b. Verify short-circuit evaluation using function calls.
 - c. Compare normal evaluation and short-circuit evaluation.
 - d. Measure the number of function calls executed in each case.
- 6. Task 6:** To study function implementation and parameter passing techniques.
 - a. Implement programs demonstrating:
 - i. Call by Value
 - ii. Call by Reference
 - b. Explain Call by Result and Call by Value-Result using examples.
 - c. Write a menu-driven calculator using functions.
- 7. Task 7:** To understand runtime exception management and event-driven programming.
 - a. Write programs for handling:
 - i. Divide-by-zero exception
 - ii. Invalid input exception
 - b. Develop a simple GUI application using Python Tkinter or Java Swing containing:
 - i. Button
 - ii. Text Box
 - iii. Label
 - c. Implement button click event handling.
- 8. Task 8:** To understand functional programming features.

- a. Implement programs using:
 - i. `lambda()`
 - ii. `map()`
 - iii. `filter()`
 - iv. `reduce()`
- b. Compare procedural and functional solutions for:
 - i. Finding square of numbers
 - ii. Filtering even numbers
- c. Explain advantages and limitations of functional programming.

9. Task 9: To explore different programming paradigms and modern languages.

- a. Demonstrate Python as:
 - i. Procedural Language
 - ii. Object-Oriented Language
 - iii. Functional Language
- b. Perform a case study of any two languages among:
 - i. Ada
 - ii. Perl
 - iii. Go
 - iv. Ruby
- c. Compare:
 - i. Features
 - ii. Advantages
 - iii. Limitations
 - iv. Application Areas
- d. Prepare a presentation (10–12 slides) summarizing findings.